

LIVRABLE 2 / 3

Proposition d'architecture

Audit de l'existant · Spécifications techniques · Choix technologiques · Diagrammes UML · Modèle de données

VERSION	DATE	AUTEUR	STATUT
2.0	Mai 2026	Architecte logiciel	Validé

Sommaire

Your Car Your Way · Application web centralisée · v2.0

1

Audit de l'existant

Analyse des 4 applications actuelles — données réelles

2

Spécifications techniques

Exigences mesurables et contraintes de déploiement

3

Architecture applicative

Monolithe modulaire + couche événementielle WebSocket

4

Modèle de données

Entités, attributs, relations — décisions de conception

5

Choix technologiques

Comparatifs Django / PostgreSQL / Redis / Channels

6

Diagrammes UML

Classes · Composants · Séquence · Déploiement (PlantUML)

7

Synthèse des décisions

Récapitulatif et cohérence avec le cahier des charges

1

Audit de l'existant

Analyse des 4 applications — données réelles issues de la description technique fournie

1.1 Critères d'évaluation

Critère	Définition	Seuil acceptable
Maintenabilité	Facilité à corriger et faire évoluer le code	Stabilisation post-release < 2 jours
Fiabilité	Disponibilité et capacité à se rétablir	Dispo ≥ 99,5 % · MTTR < 1h
Performance	Capacité à absorber la charge sans dégradation	Taux d'erreur < 1 % en pic · > 300 req/s
Sécurité	Hachage MDP, chiffrement, gestion des secrets	Hachage ≥ bcrypt · TLS ≥ 1.2
Évolutivité	Capacité à ajouter des fonctionnalités et unifier	API commune · schéma partagé
Cohérence	Homogénéité des fonctionnalités entre pays	Même expérience quel que soit le marché

1.2 Les 4 applications existantes

Application	Backend	Frontend	Hébergement
France + DE/ES/IT Base historique + dérivés copiés	Java EE	JSP / JSF	OVH (déploiements manuels)
Royaume-Uni Rachat d'un produit existant	PHP Laravel	—	AWS EC2
Canada Modernisation — résultat mitigé	Node.js	React	AWS
États-Unis Seule app containerisée — non généralisée	Spring Boot	Angular	Azure (containers)

1.3 Fiabilité

Indicateur	FR / DE / ES / IT	UK	Canada	USA
Disponibilité (12 mois)	97,2 %	98,6 %	98,1 %	98,9 %
MTTR après incident	≈ 2h45	≈ 1h10	≈ 1h10	≈ 1h10
Succès déploiements	82 % (manuel)	91 %	91 %	91 %
Stabilisation post-release	3,4 jours	1,7 jour	1,7 jour	1,7 jour

1.4 Sécurité

Indicateur	FR / DE / ES / IT	UK	Canada	USA
Hachage MDP	SHA-1 ▲ critique	bcrypt (10)	argon2id	bcrypt (12)
TLS	TLS 1.0 encore actif	Standard	Standard	Standard
Secrets	Fichiers de config serveur	Env. AWS	Env. AWS	Azure KeyVault
Vulnérabilités packages	35 à 41 %	18 %	22 %	11 %



Alerte critique — SHA-1 sur FR/DE/ES/IT

SHA-1 est cryptographiquement cassé depuis 2017. Les mots de passe du plus grand marché (Europe historique) sont stockés avec un algorithme vulnérable. Non-conformité RGPD caractérisée et risque de sécurité immédiat.

1.5 Performance et disponibilité

Indicateur	FR / DE / ES / IT	UK	Canada	USA
Indisponibilité mensuelle	21 à 28 min	9 à 16 min	9 à 16 min	7 min
Charge max sans dégradation	≈ 150 req/s	≈ 250 req/s	≈ 300 req/s	≈ 350 req/s
Taux d'erreur en pic	jusqu'à 4 %	1,5 %	1,5 %	0,8 %
Redondance	Aucune	Partielle	Partielle	Containerisée
Sauvegardes	Manuelles, non testées	AWS, non testées	AWS, non testées	Azure auto · test/90j

✓ Conclusion

La refonte est **techniquement incontournable**. L'application US démontre qu'une architecture moderne est atteignable et sert de référence. La dette technique sur FR/DE/ES/IT (SHA-1, TLS 1.0, 97,2 % de dispo) rend le statu quo impossible.

2

Spécifications techniques

Exigences mesurables et contraintes de déploiement

Exigence	Valeur cible	Outil de mesure
Temps de réponse HTTP	< 2 secondes (P95)	Tests de charge k6 / Locust
Latence WebSocket	< 500 ms	Mesure round-trip en test
Disponibilité	≥ 99,5 % / mois	Uptime Robot / Datadog
Accessibilité	WCAG 2.1 niveau AA	Axe DevTools + tests NVDA
Sécurité	OWASP Top 10 couvert	Audit pentest avant mise en prod

Contrainte déploiement	Décision	Justification
Hébergement	Cloud (AWS ou GCP)	Scalabilité, pas d'infrastructure physique
Conteneurisation	Docker	Dev = Recette = Prod — reproductibilité
Chiffrement	HTTPS + WSS obligatoires	Exigence RGPD + sécurité
Migration	Progressive — ancien système maintenu 6-12 mois	Continuité de service pour les clients actifs

3

Architecture applicative

Monolithe modulaire + couche événementielle WebSocket

3.1 Comparaison des modèles d'architecture

✗ Écarté

Microservices

- + Scalabilité granulaire
- Complexité opérationnelle très élevée
- Équipe DevOps dédiée requise
- Coût infrastructure doublé

✗ Écarté

Serverless

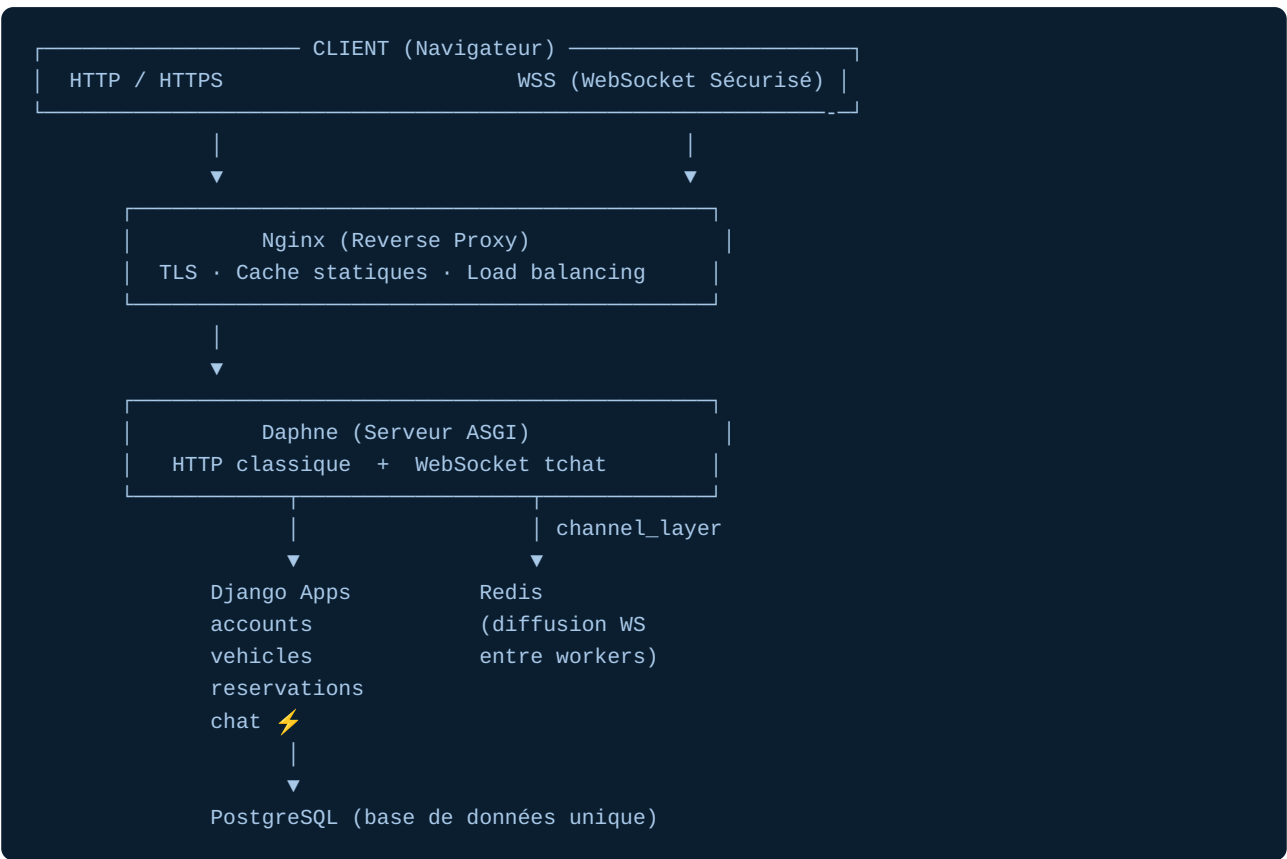
- + Pas de serveur à gérer
- WebSocket difficile en serverless
- Vendor lock-in fort
- Cold start (latence démarrage)

✓ Retenu

Monolithe modulaire

- + Déploiement simple et unique
- + Cohérence des données
- + Équipe unique, outillage unifié
- + Scalable horizontalement
- Scalabilité moins granulaire

3.2 Vue d'ensemble



3.3 Rôle de chaque composant

Composant	Rôle	Analogie
Nginx	Reverse proxy · TLS · cache statiques	Le portier qui oriente les visiteurs
Daphne	Serveur ASGI : HTTP + WebSocket simultanément	Les employés qui traitent les demandes

Composant	Rôle	Analogie
Django Channels	Extension Django pour les WebSockets du tchat	Le téléphoniste qui maintient les lignes
Redis	Channel Layer : synchronise les messages WS entre workers	Le standard téléphonique central
PostgreSQL	Base de données relationnelle principale	L'armoire à dossiers de l'entreprise

4

Modèle de données

Entités, attributs, relations et décisions de conception notables

Entité	Champs principaux	Décision notable
User	id, email (unique), first_name, last_name, language, country, is_active, is_staff	Hérite de <code>AbstractUser</code> Django — auth intégrée
Room	id, name (unique), created_at, participants (M2M → User)	M2M : room multi-agents possible
Message	id, room (FK-CASCADE), author (FK nullable-SET_NULL) , content, sent_at, is_read	RGPD : <code>author = NULL</code> à la suppression du compte (anonymisation)
Vehicle	id, brand, model, year, category, seats, transmission, fuel, price_per_day, is_available, agency (FK)	<code>is_available</code> : masquer sans supprimer
Agency	id, name, address, country, latitude, longitude, phone, is_active	<code>is_active = False</code> : fermeture temp. sans perte historique
Reservation	id, client (FK), vehicle (FK), start_date, end_date, pickup_agency, return_agency, status , total_price	<code>status</code> : pending → confirmed → completed / cancelled

5

Choix technologiques

Comparaison objective — une décision justifiée pour chaque composant

5.1 Framework backend — Django 4.2 LTS

Critère	Django 4.2 ✓	FastAPI	Node.js / Express
ORM intégré	✓ Oui	✗ Non	✗ Non
Admin back-office	✓ Auto-généré	✗ À développer	✗ À développer
Auth + i18n intégrés	✓ Oui	✗ Manuel	~ Externe
LTS sécurité	✓ Jusqu'en 2026	✗ Pas de LTS	—

5.2 Temps réel — Django Channels 4 + Redis

Critère	Django Channels + Redis ✓	Socket.io (Node)	Pusher (SaaS)
Intégration Django	✓ Native officielle	✗ Aucune	~ Via SDK
Second service	✓ Non	✗ Oui (Node.js)	✗ Oui
Données chez un tiers	✓ Non	✓ Non	✗ Oui (RGPD !)

5.3 Base de données — PostgreSQL 15

Critère	PostgreSQL ✓	MySQL	MongoDB
Schéma strict	✓ Oui	✓ Oui	✗ Non (problème audit)
Transactions ACID	✓ Complètes	✓ Complètes	~ Partielles
ORM Django natif	✓ Premium	✓ Oui	✗ Via djongo (non officiel)



Pourquoi Daphne et pas Gunicorn ?

Gunicorn est le serveur WSGI classique — il ne supporte pas les WebSockets. Dès qu'on intègre Django Channels, on passe obligatoirement en ASGI. **Daphne** est le serveur ASGI officiel de l'équipe Channels : HTTP + WebSocket simultanément.

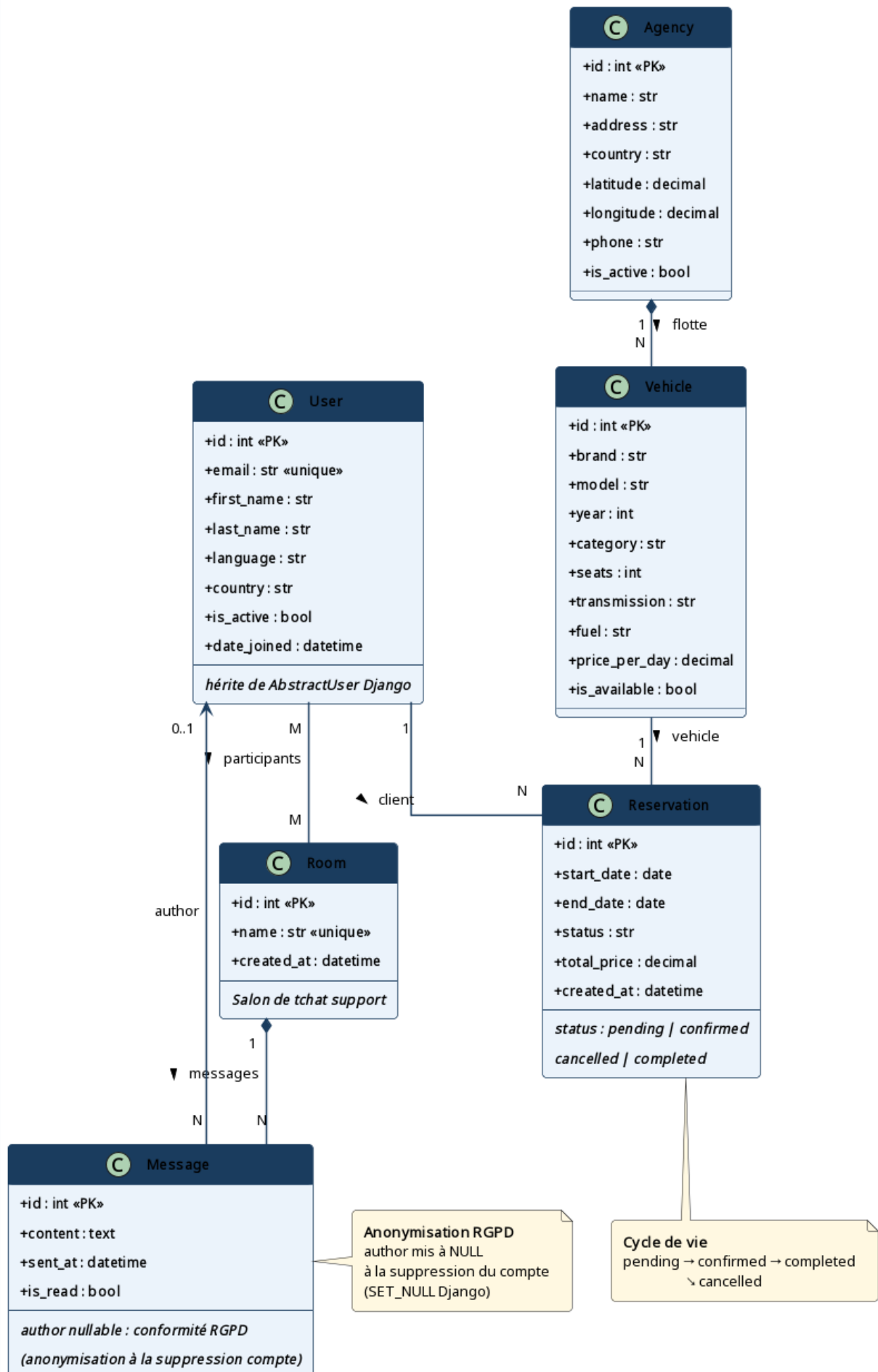
6

Diagrammes UML

Générés avec PlantUML (standard industrie) — sources dans docs/uml/

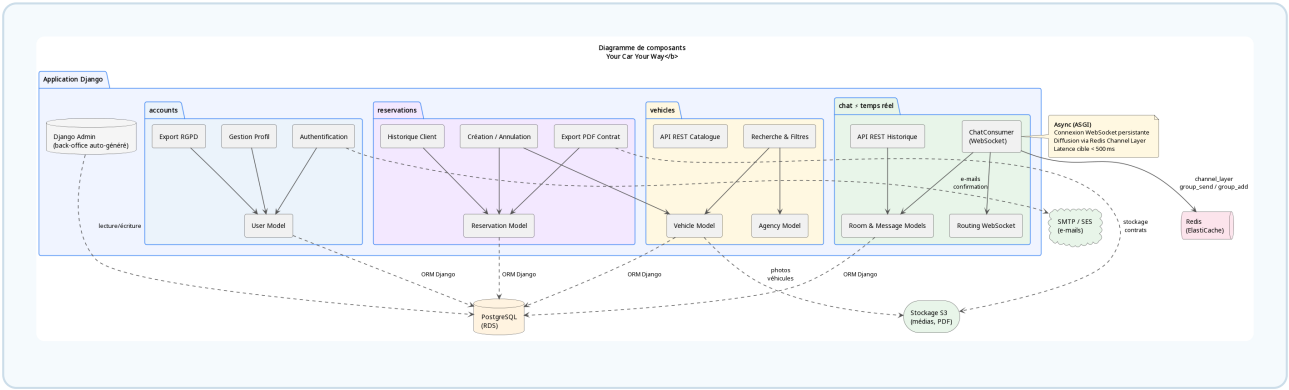
Entités persistées en base (*User*, *Room*, *Message*, *Vehicle*, *Agency*, *Reservation*), attributs et relations. Vue « données » de l'application.

Diagramme de classes — Modèle de données
Your Car Your Way



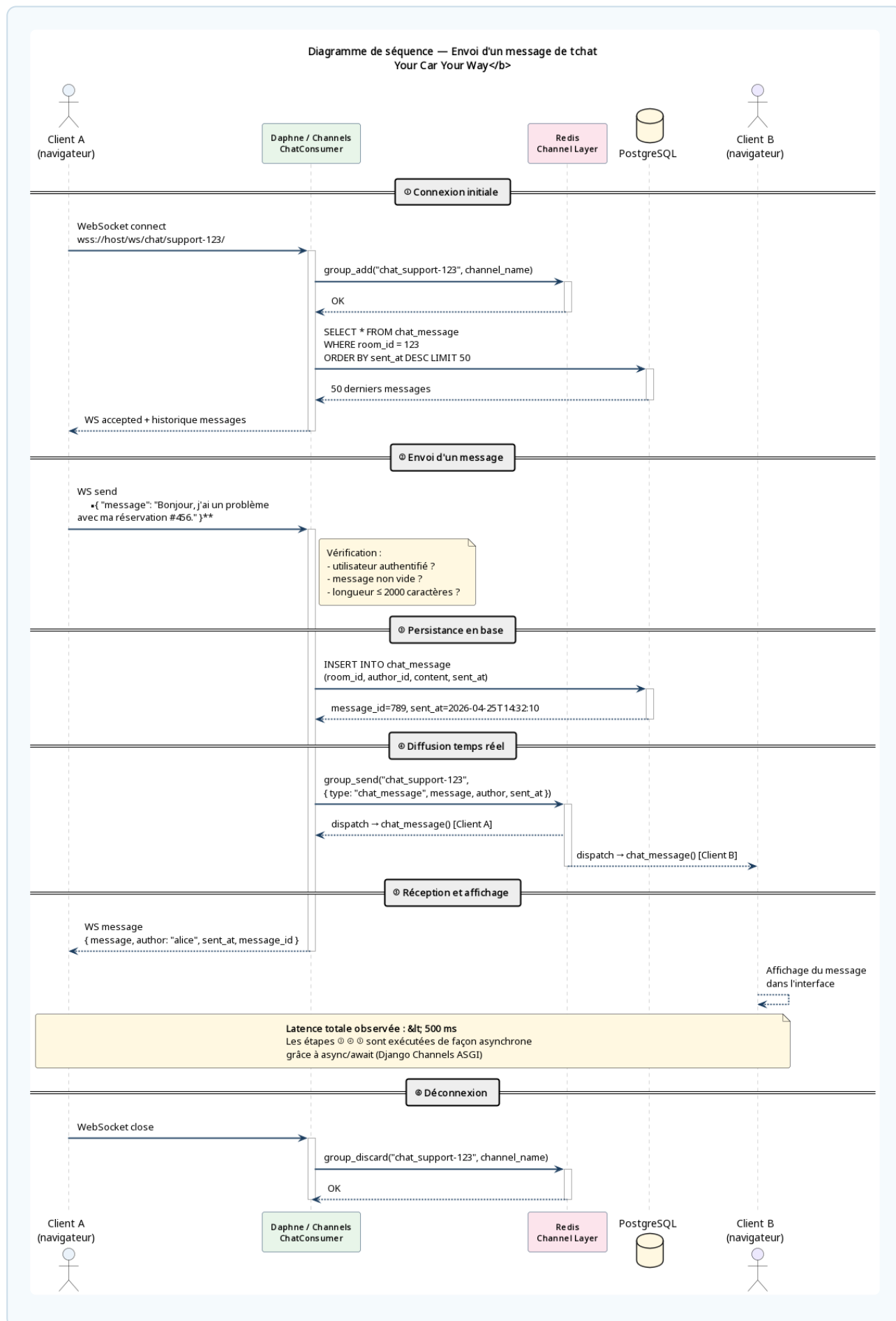
Modules Django (accounts, vehicles, reservations, chat) et dépendances vers l'infrastructure (PostgreSQL, Redis, S3, SES).

Vue « organisation ».

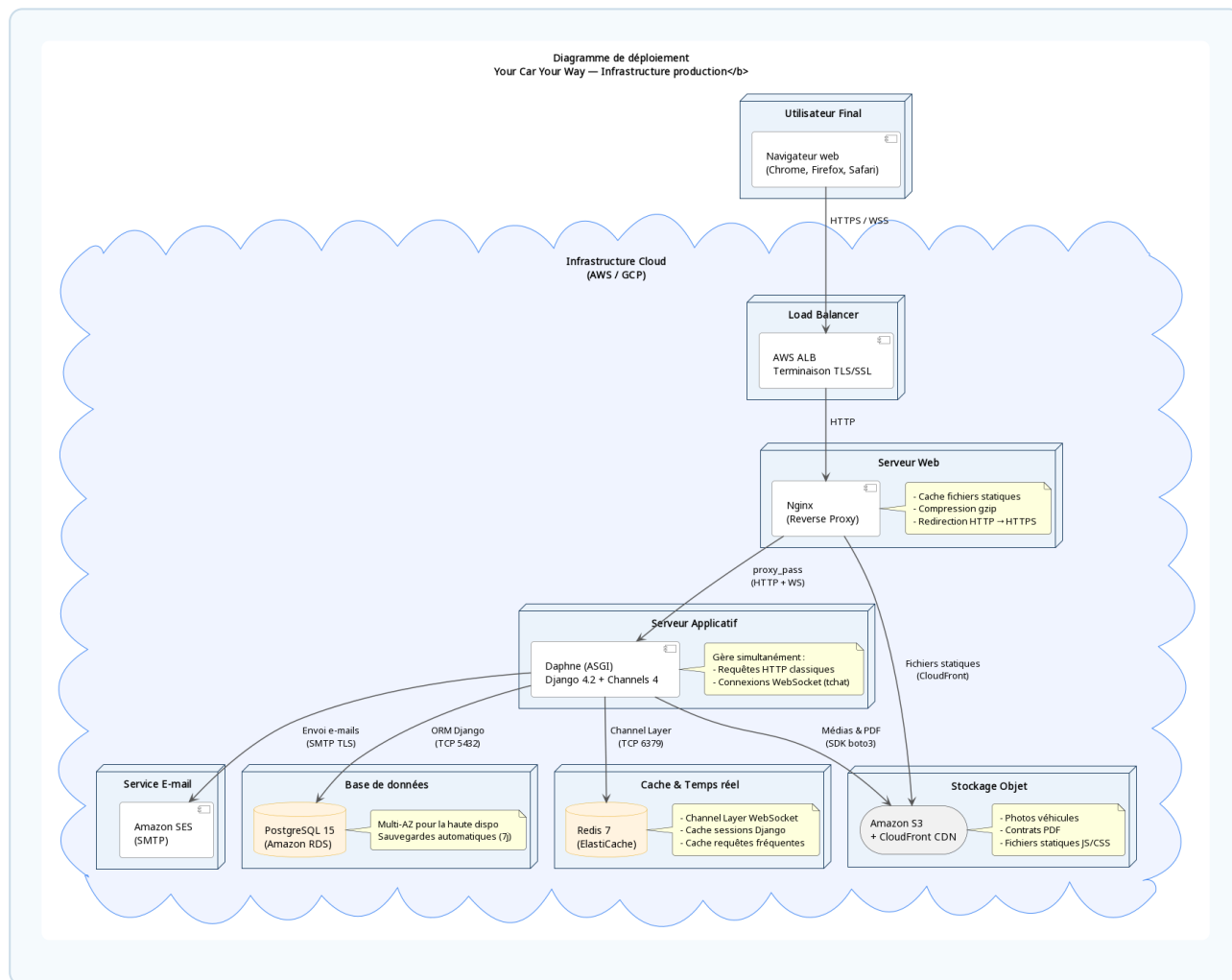


6.3 — DIAGRAMME DE SÉQUENCE · FLUX ENVOI D'UN MESSAGE

6 étapes chronologiques : connexion WS → persistance PostgreSQL → diffusion Redis → réception. Latence totale < 500 ms.



Nœuds de production : Nginx (TLS) → Daphne (ASGI) → PostgreSQL / Redis / S3 / SES. Architecture cloud haute disponibilité.



7

Synthèse des décisions

Récapitulatif et cohérence avec le cahier des charges

Décision	Choix retenu	Alternative écartée	Raison principale
Architecture	Monolithe modulaire	Microservices	Complexité adaptée, évolutif si besoin
Framework	Django 4.2 LTS	FastAPI, Node.js	ORM, admin, auth, i18n — tout intégré
Temps réel	Channels 4 + Redis	Socket.io	Intégration Django native, pas de second service
Base de données	PostgreSQL 15	MongoDB	Schéma strict, ACID, ORM natif
Serveur	Daphne (ASGI)	Gunicorn (WSGI)	WebSocket impossible en WSGI
API	REST (DRF)	GraphQL	Standard, outillage mature
Langage	Python 3.11+	PHP, JavaScript	Un seul langage, écosystème IA/data

Cohérence avec le cahier des charges

Exigence fonctionnelle	Réponse technique
Compte unique cross-marchés	Base PostgreSQL unique, modèle <code>User</code> centralisé
Tchat temps réel < 500 ms	Django Channels + WebSocket + Redis Channel Layer
Accessibilité WCAG 2.1 AA	HTML sémantique, <code>aria-live</code> , <code>role="log"</code> dans les templates
Multilingue FR/EN/ES	i18n natif Django — fichiers <code>.po</code> / <code>gettext</code>
RGPD — effacement	<code>author</code> nullable (<code>SET_NULL</code>) + endpoint export JSON
Back-office administrateur	Django Admin auto-généré avec <code>ModelAdmin</code>